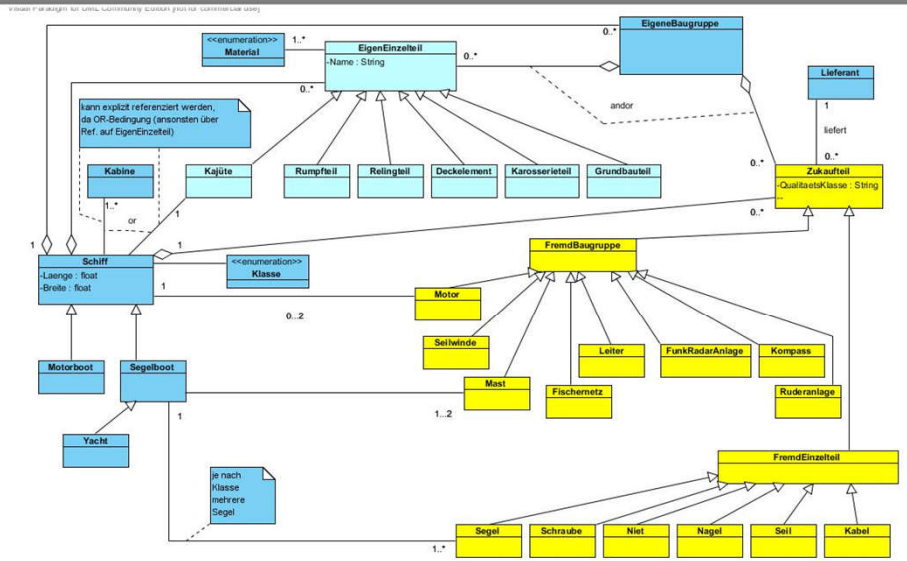
Karlsruher Institut für Technologie

DHBW Duale Hochschule Baden-Württemberg

Software Engineering I

Grundlagen der Objektorientierung
Einführung in die objektorientierte Datenmodellierung

Institut für Angewandte Informatik (IAI)



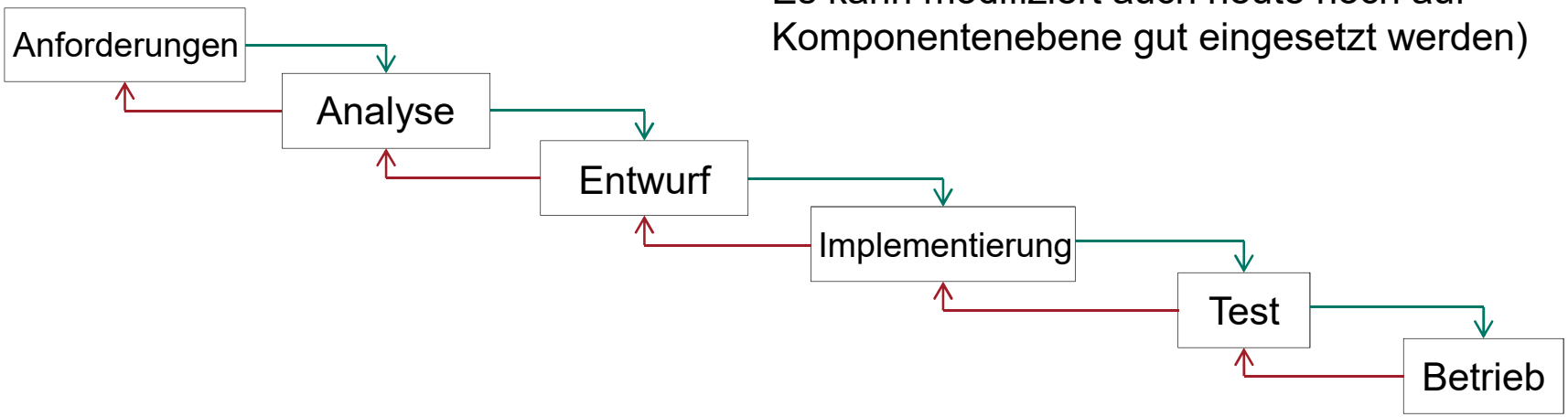
Historie

- Objektorientierte Programmierung
  - Seit den späten 50er bzw. frühen 60er Jahren bekannt (LISP, Simula 67, Smalltalk, später C++, C#, Java, …)
  - Beinhaltet Attribute, Funktionen sowie Verbindungen zu anderen Objekten

- Objektorientierte Modellierung
  - 1990: OOD (Object Oriented Design, Booch)
  - 1992: OMT (Object Modeling Technique, Rumbaugh)
    OOA (Object Oriented Analysis, Coad/Yourdon)
    OOSE (Object Oriented Software Engineering, Jacobsen)
  - 1997: UML (Unified Modeling Language, Booch/Jacobsen/Rumbaugh), standardisiert durch OMG (Object Management Group)
  - 2004: UML 2.0

Softwareentwicklungs-Prozess

- Jede Softwareentwicklung beinhaltet eine gründliche **Analyse** der Problemstellung!
  - Teilweise drastische Reduzierung von Zeit und Komplexität bei Entwurf, Implementierung und Test der späteren Software.
  - Analyse führt zur Identifikation der beteiligten Elemente und Zusammenhänge (Klassen, Objekte und Relationen)
- „Ur-Modell“ ist das Wasserfallmodell nach Royce und Böhm
  (bildet die Grundlage fast aller weiteren Vorgehensmodelle der Softwareentwicklung.
  Es kann modifiziert auch heute noch auf Komponentenebene gut eingesetzt werden)



Was ist ein Objekt?

Karlsruher Institut für Technologie

- allgemein: eine Sache oder ein Gegenstand
- eine Einheit beim Programmieren
- eine Einheit beim Modellieren, d.h. Analysieren und Entwerfen
- ein Objekt kann konkret oder auch konzeptuell sein:

konkret

Datei in Dateisystem, Hund, Katze, Person, Haus, Computer, …

konzeptuell

Zuteilungsstrategie in einem Multiprozessor-Betriebssystem,
Buchung eines Flugs oder eines Zimmers,
Kauf eines Gegenstands, usw.

Was ist Objektorientierung?

- Methodik und Vorgehensweise beim Erstellen von Softwareprodukten, bei denen Objekte und deren Zusammenhänge im Mittelpunkt stehen
- Objekte vereinen Daten(-struktur) **und** Verhalten (Funktionalität) in sich
- Gegensatz:
  konventionelle, *prozedurale SW-Entwicklung (Programmierung)*
  - Funktionen und Strukturen stehen im Mittelpunkt
  - Datenstruktur und Verhalten sind nur lose miteinander verbunden

Was ist objektorientierte Modellierung?

Karlsruher Institut für Technologie

- Möglichst genaue Abbildung eines Problembereichs der realen Welt in die Modellwelt.
- Wird für die Analyse des Problemumfelds und für den Entwurf der Software verwendet (A & E stehen im SW-Entwicklungsprozess grundsätzlich **vor** der Implementierung)
- Ab einer bestimmten Komplexität des Problemumfelds unerlässlich (vermeidet fehleranfälliges „happy hacking“)
- Ein Teil des Modells kann direkt in Quellcode umgewandelt werden.

Was ist der Vorteil von Objektorientierung?

- Aufgaben, Strukturen, Eigenschaften (Attribute) können klar getrennt und separat behandelt werden (Übersichtlichkeit)
- Gemeinsamkeiten können gebündelt werden (Vereinfachung)
- Ergebniselemente (Klassen) können leichter in anderen SW-Produkten verwendet werden

➡ **hohe Wiederverwendbarkeit!**

Objekte, Klassen, Instanzen, Methoden, Operationen ???

Karlsruher Institut für Technologie

- Objekte (alternativ: Instanzen):
  - reale Elemente in Alltag, Problemumfeld und Programm
- Klassen:
  - Beschreibung (Definition, Schablone) eines Objekts.
  - Eine Klasse besteht aus Attributen (Eigenschaften bzw. Daten) und Methoden (Verhalten)
- Operation:
  - Beschreibung des Verhaltens von Objekten einer Klasse
- Methode:
  - Realisierung (Implementierung, Quellcode) einer Operation
  - Wird oft auch als alternativer Begriff für Operation verwendet

Herausforderung

- Wie kommt man von der realen Welt in die Modellwelt?
- Wie findet man die passenden Objekte, Klassen und Attribute?

???

Software

Klassen

Referenzen

Attribute

Funktionalität

Herausforderung 1: Identifizieren der Klassen

Laptop

Boot

Tasse

Herausforderung 2: Identifizieren der Attribute

- Attribute beschreiben die charakteristischen Eigenschaften einer Klasse

<u>Laptop:</u>
Bildschirmgröße, Farbe, Höhe, Breite, Tiefe, Ausstattung, …
an/aus, offen/zu, am Netz / nicht am Netz, …

<u>Boot:</u>
Form, Farbe, Höhe, Länge, Gewicht, Breite, Platzanzahl, Material, …
fahrend/stehend, leer/besetzt, …

<u>Tasse:</u>
Form, Farbe, Höhe, Aufdruck, Durchmesser, Material, …
leer/voll, heiß/kalt, bruchfest/zerbrechlich, …

Herausforderung 3: Identifizieren der Funktionalität

- Grundfrage: was macht das Objekt, wenn es „dazu beauftragt“ wird?

Laptop:
starten, herunterfahren,
Programm ausführen,
mit Netz verbinden, …

Boot:
losfahren, anhalten, beschleunigen,
wenden, …

Tasse:
???, …

Herausforderung 4: Identifizieren assoziierter Klassen

- Grundfrage: zu welchen anderen Objekten gibt es Verbindungen?

<u>Laptop:</u>
Tastatur, Festplatte, CPU, Touchpad, Bildschirm, Netzverbindung, Besitzer, …

<u>Boot:</u>
Motor, Sitze, Lenkrad, Kajüte, Fahrer, Besitzer, Mitfahrer, …

<u>Tasse:</u>
Besitzer, Deckel, Inhalt, …

Herausforderung 5: Identifizieren von Gemeinsamkeiten

- Grundfrage: welche Gemeinsamkeiten und Unterschiede gibt es für „gleichartige“ Objekte?

PKW:
Hersteller, Modell, Höhe, Breite, Länge, Leistung, Verbrauch, Gewicht, Farbe, an/aus, offen/zu, Sitzplätze, …
fahren, stehen, tanken, beschleunigen, …
➔ Motor, Räder, Sitze, Besitzer, **Faltdach**, …

LKW:
Hersteller, Modell, Höhe, Breite, Länge, Leistung, Verbrauch, Gewicht, Farbe, an/aus, offen/zu, Sitzplätze, …
fahren, stehen, tanken, beschleunigen, …
➔ Motor, Räder, Sitze, Besitzer, **Kran, Seilwinde,** …

Bus:
Hersteller, Modell, Höhe, Breite, Länge, Leistung, Verbrauch, Gewicht, Farbe, an/aus, offen/zu, Sitzplätze, …
fahren, stehen, tanken, beschleunigen, …
➔ Motor, Räder, Sitze, Besitzer, **Mikrofon**, …

Wie werden die Objekte geeignet dargestellt?

Karlsruher Institut für Technologie

- Objektdiagramm, Instanzendiagramm
  - Formale grafische Notation, um real vorhandene **Objekte** und deren Relationen zu anderen Objekten darzustellen.
  - Es können beliebig viele Objekte desselben Typs (Klasse) existieren.
  - Besonders nützlich, um Testfälle (vor allem Szenarien) zu dokumentieren und Beispiele zu diskutieren.

- Klassendiagramm
  - Grafische Notation, um **Klassen** und deren Relationen zu anderen Klassen darzustellen.
  - Schema, Muster oder Template (Schablone) zur Beschreibung vieler möglichen Objektinstanzen. Ein Klassendiagramm beschreibt Objektklassen.
  - Jede Klasse ist nur einmalig vorhanden.
  - Ein gegebenes Klassendiagramm entspricht einer unendlichen Menge von Instanzendiagrammen

Die 5 wichtigsten Aspekte der Objektorientierung

Karlsruher Institut für Technologie

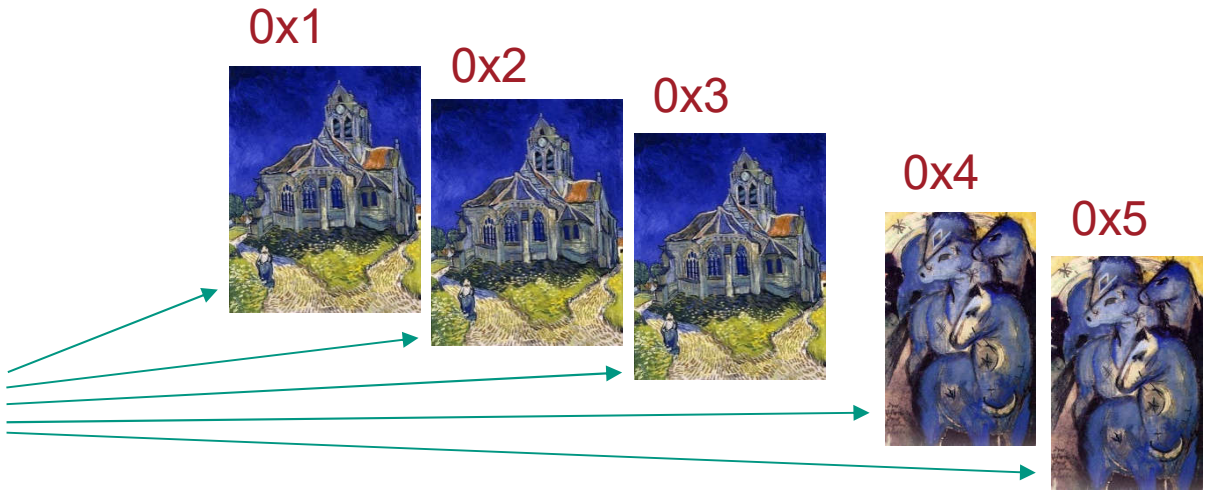
- Identität
- Klassifikation
- Vererbung
- Polymorphie (Polymorphismus)
- Kapselung

Die 5 wichtigsten Aspekte der OO: 1.) Identität

- Daten werden diskreten, unterscheidbaren Entitäten (Objekten) zugeordnet
- Jedes Objekt besitzt eine eigene inhärente ("ihm innewohnende") Identität, d.h. zwei Objekte sind klar voneinander unterscheidbar, selbst wenn alle ihre Attributwerte (wie Name oder Größe) identisch sind
- Jedes Objekt hat in einer Programmiersprache einen eindeutigen "*Anfasser*" (z.B. Speicheradresse „0xN“) ➔ ermöglicht eindeutigen Zugriff

Klasse: ***Bild***

Attribute: Titel
Name d. Künstlers
Breite
Höhe
Stilrichtung



Die 5 wichtigsten Aspekte der OO: 1.) Identität (2)

Karlsruher Institut für Technologie

- Objektzugriffe
  - sind einheitlich und unabhängig vom Inhalt der Objekte
  - erlauben es, gemischte Objektkollektionen zu erzeugen (z.B. durch ein Dateisystem-Verzeichnis, das sowohl Dateien als auch Unterverzeichnisse enthält). ➔ siehe *Polymorphie*

Java-Beispiel:

```
public class Hundeschule{
    . . .
    Hund fiffi = new Hund(„Fiffi");
    Hund bello = new Hund(„Bello");

    bello.machMaennchen();
    fiffi.machMaennchen();
    bello.sitz();
    fiffi.sitz();
    . . .
}
```

Die 5 wichtigsten Aspekte der OO: 2.) Klassifikation

- Objekte mit der gleichen Datenstruktur (***Attribute***) und dem gleichen Verhalten (***Operationen, Methoden***) können zu einer ***Klasse*** gruppiert werden

Landfahrzeug

Wasserfahrzeug

Fahrzeug

Die 5 wichtigsten Aspekte der OO: 2.) Klassifikation (2)

- Eine ***Klasse*** ist eine Abstraktion, die die Eigenschaften beschreibt, die für eine Anwendung wichtig sind, und den Rest ignoriert.
- Die Wahl von Klassen ist immer beliebig und hängt von der Anwendung ab
- Jede Klasse beschreibt eine unendliche Menge individueller Objekte.
- Jedes Objekt (Instanz) der Klasse besitzt eigene **Werte** für alle ihre Attribute, während sie die Attributnamen und Operationen mit anderen Instanzen der Klasse teilt.

Klasse: ***Bild***

Attribute: Titel
Künstler
Breite
Höhe
Stil

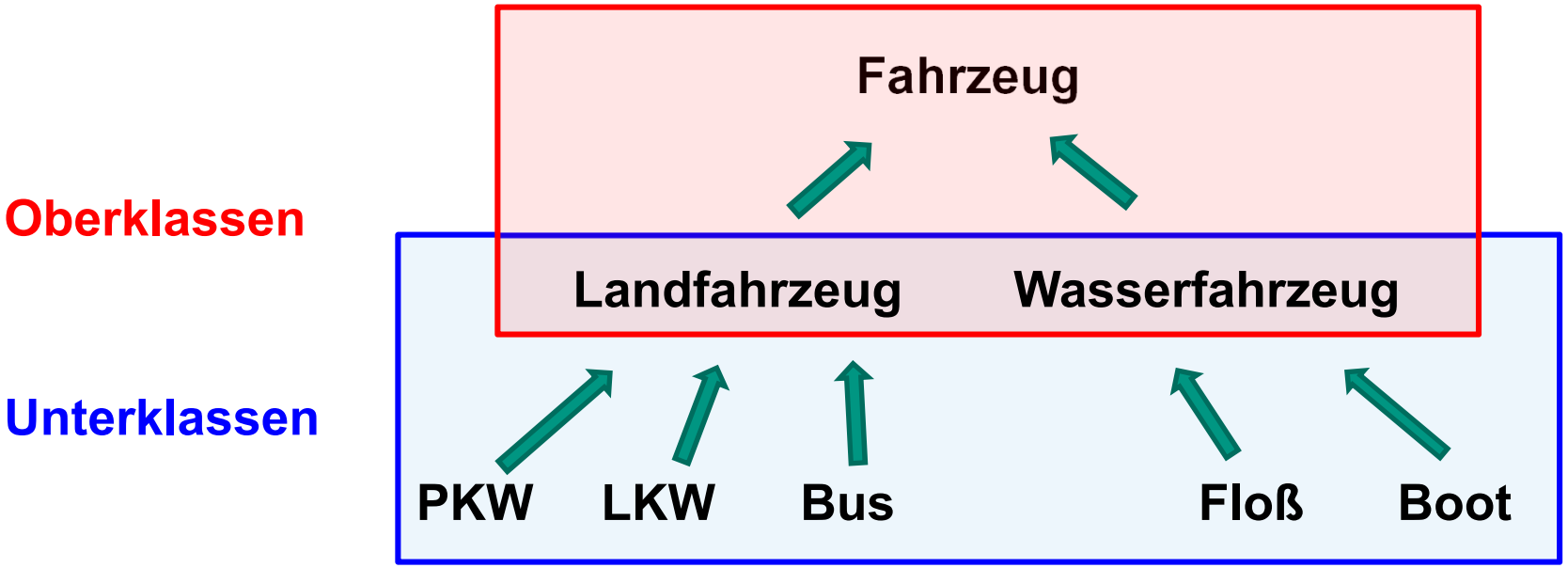
Titel='Kirche von Auvers'
Künstler='Vincent van Gogh'
Breite=74
Höhe=94
Stil='Impressionismus'

Titel='Turm der blauen Pferde'
Künstler='Franz Marc'
Breite=130
Höhe=200
Stil='Expressionismus'

Titel='Mona Lisa'
Künstler='Leonardo da Vinci'
Breite=53
Höhe=77
Stil='Renaissance'

Die 5 wichtigsten Aspekte der OO: 3.) Vererbung

- Mehrere Klassen mit denselben Attributen und/oder demselben Verhalten können durch eine gemeinsame Oberklasse repräsentiert werden (➔ *Generalisierung*, v.a. in Analysephase)
- Eine Klasse kann sehr allgemein definiert sein und dann in immer detailliertere Unterklassen verfeinert werden (➔ *Verfeinerung*, v.a. in Entwurfsphase)



Die 5 wichtigsten Aspekte der OO: 3.) Vererbung (2)

Karlsruher Institut für Technologie

- Jede Unterklasse übernimmt oder erbt alle Eigenschaften und Methoden ihrer Oberklasse und fügt ihre eigenen individuellen Eigenschaften und Methoden hinzu

```
public class Oberklasse{
    . . .
    public int OberklassenAttribut;
    public void doSomething(){..}
}
```

```
public class Unterklasse
          extends Oberklasse{
    public int UnterklassenAttribut;
    public void doItYourself()
}
```

geerbt

eigen

```
public class IrgendeineKlasse{
    public Unterklasse ukl;
    . . .
    ukl = new Unterklasse();

    ukl.doSomething();
    ukl.OberklassenAttribut=5;

    ukl.doItYourself();
    ukl.UnterklassenAttribut=4;
    . . .
}
```

Die 5 wichtigsten Aspekte der OO : 4.) Polymorphie

Karlsruher Institut für Technologie

Polymorphie tritt immer im Zusammenhang mit Vererbung und Schnittstellen, d.h. mit Oberklassen und Interfaces auf

Die 2 wichtigsten Arten von Polymorphie:

- Polymorphie von Datentypen oder Klassen
- Polymorphie einer Methode bzw. Operation

Die 5 wichtigsten Aspekte der OO: 4.) Polymorphie

Polymorphie von Datentypen oder Klassen:

- Eigenschaft einer Variablen, für Objekte verschiedener Klassen stehen zu können („Platzhalter“)

Deklaration:

```
public class Oberklasse{
    . . .
    public void doSomething(){
        . . .
    }
}
```

```
public class UnterklasseEins
        extends Oberklasse{…}
```

```
public class UnterklasseZwei
        extends Oberklasse{…}
```

Erzeugung bzw. Verwendung:

```
public class IrgendeineKlasse{
    public Oberklasse okl;
    . . .
    okl = new UnterklasseEins();
    okl.doSomething();
    . . .
    okl = new UnterklasseZwei();
    okl.doSomething();
    . . .
}
```

Die 5 wichtigsten Aspekte der OO: 4.) Polymorphie

Karlsruher Institut für Technologie

Polymorphie einer Methode bzw. Operation:

- Eine **Methode** ist **polymorph**, wenn sie in verschiedenen Klassen die gleiche Spezifikation hat, jedoch in der jeweiligen Klasse individuell implementiert ist (so genanntes *Überschreiben*)

```
public class Oberklasse{
    . . .
    public void doSomething(){
        . . .
    }
}
```

```
public class UK1 extends Oberklasse{
    . . .
    public void doSomething(){
      // spezifische Impl. für UK1
    }
    . . .
}
```

```
public class UK2 extends Oberklasse{
    . . .
    public void doSomething(){
      // spezifische Impl. für UK2
    }
    . . .
}
```

Die 5 wichtigsten Aspekte der OO: 5.) Kapselung

Karlsruher Institut für Technologie

- Unter Kapselung versteht man das Verbergen von Implementierungsdetails, Daten oder Informationen vor dem Zugriff von außen.
- Der direkte Zugriff auf interne Daten(-struktur) wird unterbunden und erfolgt stattdessen ausschließlich über definierte Schnittstellen („Black-Box-Modell“, „kontrollierbare“ *set*-Methoden)
- Objekte können den Zustand anderer Objekte *nicht direkt* unerlaubt lesen oder ändern. Das kann über so genannte Sichtbarkeitskriterien gesteuert werden (*private, public, protected,* etc.)

```
public class AnyClass {
. . .
public int anAttribute = 0;
. . .
```

```
public class CallingClass{
. . .
AnyClass ac = new AnyClass(…);
ac.anAttribute = 123;
. . .
```

Besser!

```
public class AnyClass {
. . .
private int anAttribute = 0;
. . .
public void setAnAttribute( int val ){
  this.anAttribute = val;
  log("anAttribute changed to "+val);
}
```

```
. . .
AnyClass ac = new AnyClass(…);
ac.setAnAttribute( 123 );
. . .
```

Zusammenfassung

- Erläuterung der Begriffe:
  - Objektorientierung: Objekt, Klasse, Instanz, Attribut, Operation, ..
  - Identifizieren von Objekten und Klassen
- Erklärung der 5 wichtigsten Aspekte der Objektorientierung
  - Identität
  - Klassifikation
  - Vererbung
  - Polymorphie (Polymorphismus)
  - Kapselung